

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научный руководитель проекта, доцент
департамента программной инженерии,
факультета компьютерных наук

_____ Е. Ю. Песоцкая
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»
старший преподаватель департамента
программной инженерии

_____ Н.А. Павлочев
«__» _____ 2026 г.

Приложение для Android "Организер лекарств"

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 81 01-1-ЛУ

Исполнитель
студент группы БПИ-245
_____ / Н. Волошин /
«__» _____ 2026 г.

Москва 2026

УТВЕРЖДЕН
RU.17701729.06.05-01 81 01-1-ЛУ

Приложение для Android "Органайзер лекарств"

Пояснительная записка

RU.17701729.06.05-01 81 01-1

Листов 45

Инв. № подл	Подп. и дата
Взам. инв. №	Инв. № дубл.
Подп. и дата	Подп. и дата
Инв. № подл	Инв. № дубл.

Москва 2026

АННОТАЦИЯ

Настоящий программный документ представляет собой пояснительную записку к программному проекту «Приложение для Android «Органайзер лекарств»» и содержит следующие разделы: «Введение», «Назначение и область применения», «Технические характеристики», «Ожидаемые технико-экономические показатели», «Описание применения генеративной модели», приложения.

Раздел «Введение» включает наименование программы и документ, на основании которого ведётся разработка, с указанием организации, утвердившей этот документ.

В разделе «Назначение и область применения» приведены функциональное и эксплуатационное назначение программы и краткая характеристика области её применения.

Раздел «Технические характеристики» содержит постановку задачи на разработку программы, описание функционирования программы и её частей, описание и обоснование выбора состава технических и программных средств, описание и обоснование архитектуры клиентской и серверной частей, описание работы с базами данных, описание и обоснование выбора метода организации входных и выходных данных, а также описание особенностей функционала.

В разделе «Ожидаемые технико-экономические показатели» приведены предполагаемая потребность в программе и её экономические преимущества по сравнению с отечественными и зарубежными аналогами.

Раздел «Описание применения генеративной модели» содержит цели применения генеративных моделей при разработке программы и подготовке документации, наименования применявшихся моделей, адреса их размещения в сети «Интернет» и способы применения.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.404-79 [7]: Пояснительная записка. Требования к содержанию и оформлению.

Изменения к настоящей пояснительной записке оформляются согласно ГОСТ 19.603-78 [8], ГОСТ 19.604-78 [9].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Документы, на основании которых ведётся разработка	4
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения программы	6
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	7
3.1. Постановка задачи на разработку программы	7
3.2. Описание функционирования программы	8
3.3. Описание и обоснование выбора состава технических и программных средств	8
3.3.1. Состав технических и программных средств	8
3.3.2. Обоснование выбора технических и программных средств	9
3.3.3. Почему состав серверных средств изменился по ходу работы	10
3.4. Описание и обоснование архитектуры программы	11
3.4.1. Общее устройство	11
3.4.2. Архитектура Android-приложения	12
3.4.3. Архитектура серверной части	13
3.4.4. Приватность как архитектурное ограничение	14
3.4.5. Доступ: чтение и есть проверка	14
3.4.6. Конкурентность и целостность	15
3.4.7. Синхронизация клиента и сервера	17
3.4.8. Описание работы с базами данных	17
3.5. Описание и обоснование выбора метода организации входных и выходных данных	18
3.6. Особенности функционала	19
3.6.1. Первый запуск и учётная запись устройства	19
3.6.2. Аптечки и их содержимое	20
3.6.3. Карточка упаковки	21
3.6.4. Заведение упаковки и сканирование кода	22
3.6.5. Курсы лечения и их обеспечение	24
3.6.6. План дня, приёмы и напоминания	26

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.6.7. Общая аптечка	28
3.6.8. Отчёты	30
3.6.9. Настройки и разрешения	33
3.7. Организация разработки	34
3.7.1. Два уровня планирования	34
3.7.2. Прослеживаемость изменений	35
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	36
4.1. Предполагаемая потребность	36
4.2. Экономические преимущества разработки по сравнению с аналогами	36
5. ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ	39
5.1. Цели применения	39
5.2. Применявшиеся модели	39
5.3. Способ применения и разграничение ответственности	39
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	41
ПРИЛОЖЕНИЕ 2. ТЕРМИНОЛОГИЯ	43

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы: «Приложение для Android «Организер лекарств»».

Наименование программы на английском языке: «Medicine Organizer» Android Application.

Краткое наименование на английском языке: MedApp.

1.2. Документы, на основании которых ведётся разработка

Разработка ведётся на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утверждённой академическим руководителем программы темы курсового проекта.

Требования к программе установлены техническим заданием «Приложение для Android «Организер лекарств». Техническое задание» RU.17701729.06.05-01 ТЗ 01-1.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Функциональное назначение

Программа предназначена для решения следующих задач:

1. Учет препаратов: добавление, просмотр, редактирование данных о препаратах (количество, срок годности и т.п.).
2. Учёт аптек: человек заводит и редактирует (условно) неограниченное число отдельных аптек и присваивает каждому препарату место хранения.
3. План лечения и напоминания: человек указывает, когда и в какие периоды ему следует принимать препарат. На основании этого, сроков годности и остатков ведётся учёт медикаментов.
4. Совместное использование аптечки - пользователь может позволить иному пользователю пользоваться его аптечкой.
5. Поиск по всем аптечкам и по каждой в отдельности, фильтрация и сортировка.
6. Аналитика: расчёт расхода препаратов на дату в будущем, история израсходованного за период, сводная статистика о препаратах и сводный план лечения на дату.

Программа не назначает и не контролирует лечение, а обеспечивает организацию сведений и напоминания. Она не является медицинским изделием и не подлежит соответствующей сертификации.

2.2. Эксплуатационное назначение

Программа предполагает повседневное использование на смартфоне под управлением операционной системы Android версии 10 и выше. Детально обработаны случаи совместного (общая аптечка) и распределённого (несколько аптек) использования лекарств.

Предполагаются следующие условия эксплуатации:

- платформа: смартфон, выпущенный не ранее 2022 года, под управлением Android 10+;
- режим использования: ежедневный, возможны периоды длительного простоя;
- сетевые условия: базовая функциональность доступна без сети; для работы совместных аптек предполагается стабильное интернет-соединение. Система может некоторое время поддерживать работу без него и синхронизироваться при появлении, но это может создавать конфликты и откаты некоторых операций;
- использование системных возможностей: показ уведомлений, точные будильники, доступ к камере для сканирования кодов маркировки;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- ограничения и допущения: своевременность напоминаний зависит от настроек операционной системы, а согласованность совместных аптек - от доступности сети и возможности фоновой синхронизации. Успешность работы функции общего использования одного препарата несколькими людьми зависит от согласованности их действий и обеспечений необходимых условий для бесперебойной работы программы.

2.3. Краткая характеристика области применения программы

Контроль состояния домашних аптек представляет собой трудоёмкую задачу. По данным Всемирной организации здравоохранения и современных обзоров, в развитых странах в среднем половина пациентов с хроническими заболеваниями принимает лекарства так, как предписано врачом. Одной из самых частых причин отклонения называют забывчивость: человек пропускает приём, путает дозу или не успевает вовремя пополнить запас.

Положение усугубляет явление «общей аптечки», когда одни и те же лекарства расходуются несколькими членами семьи. По данным опросов, у подавляющего большинства российских домохозяйств есть общая домашняя аптечка с базовыми препаратами. В таких условиях трудно установить, что есть дома, в каких дозах препарат принимают разные люди и чей запас близок к исчерпанию. Часть лечения при этом распределена между несколькими местами хранения - дом, дача, работа, - что повышает риск пропусков и ошибок.

Областью применения программы является органайзер таких сведений: учёт физических упаковок, их сроков и остатков, планирование приёмов, напоминания и совместный доступ к аптечке.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Требуется разработать программу, которая ведёт учёт домашних аптечек на устройстве человека, работает без постоянного подключения к сети и позволяет нескольким людям пользоваться одной аптечкой согласованно.

Ключевым понятием предметной области выбрана **физическая упаковка**, а не лекарство как таковое. Купленная пачка не пополняется: если человек покупает такое же лекарство, заводится вторая упаковка со своим сроком годности, своей ценой и своим остатком. Такое решение отражает действительность: две пачки одного препарата имеют разные сроки годности, и объединение их в одну запись сделало бы учёт сроков невозможным.

За этим стоит и более общая трудность: что считать «одним лекарством». Одно действующее вещество, одно торговое название, одно название при разных комплектациях или одно название при разных производителях дают четыре несовпадающих ответа, и ни один из них не позволяет однозначно определить остаток и срок годности. Поэтому принято самое узкое определение: единицей учёта является конкретная физическая упаковка.

Из этого следуют такие постановки:

1. **Приём списывает из конкретной упаковки**, а не из абстрактного запаса. Плановая доза курса лечения берётся из одной упаковки и между упаковками не делится.

2. **Курс лечения обеспечивается упаковками-источниками**. Человек указывает, из каких упаковок берутся дозы и в каком порядке они расходуются. Если запас не покрывает назначенное число доз, программа сообщает о нехватке, но расписание лечения не сокращает самовольно.

3. **Работа с подключением к сети - основной режим**. Местная аптечка существует только на устройстве и подключения не требует, а действия с общей аптечкой, совершённые без сети, накапливаются в очереди операций и отправляются позже с сохранением порядка. Однако накопление неизбежно усложняет согласование: чем дольше устройство не выходило на связь, тем выше вероятность, что часть операций будет отклонена сервером и потребует пересчёта. Меры, снижающие эту вероятность, описаны в п. 3.4.7, но полностью её не устраняют, поэтому режим без сети считается временным, а не основным.

4. **Совместный доступ без персональных данных**. Сервер не должен знать о человеке ничего, кроме идентификатора устройства и хеша его ключа. Это требование ограничивает архитектуру серверной части сильнее, чем любое функциональное требование, и подробно разобрано в п. 3.4.4.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. Напоминания должны переживать перезагрузку устройства и длительный простой программы.

3.2. Описание функционирования программы

Программа состоит из трёх самостоятельных частей: Android-приложения, серверной части и вспомогательного скраппера справочника лекарственных препаратов.

Android-приложение - то, с чем работает человек. Оно хранит аптечки и упаковки в собственной базе данных на устройстве, показывает план приёмов, напоминает о них, сканирует коды маркировки и обменивается с сервером сведениями об общих аптечках. Без сети приложение полностью работоспособно во всём, что касается местных аптек (кроме изначальной обязательной регистрации).

Серверная часть - REST-сервис, который хранит общие аптечки, брони под курсы лечения и справочник лекарственных препаратов. Сервер является источником истины по количеству препарата в общей аптечке: именно он разрешает конкурентные изменения, приходящие от разных людей.

Скраппер справочника - одноразовая программа, которая собирает справочник лекарственных препаратов и готовит его к загрузке в базу данных сервера. Справочник наполняется мимо приложения, и ни клиент, ни сервер в его сборе не участвуют.

Обмен между клиентом и сервером идёт по протоколу HTTP с использованием формата JSON. Описание контракта в формате OpenAPI порождается самой серверной программой, поэтому клиент опирается на описание, которое не может разойтись с действительным поведением сервера.

3.3. Описание и обоснование выбора состава технических и программных средств

3.3.1. Состав технических и программных средств

Для работы клиентской части необходимо Android-устройство под управлением операционной системы Android 10 и выше, выпущенное не ранее 2022 года, для работы функции сканирования кодов маркировки и кодов приглашения необходима камера.

Для работы серверной части необходим виртуальный сервер под управлением операционной системы Linux с установленными Docker Engine версии 24 и выше и СУБД PostgreSQL версии 15 и выше. Минимальная конфигурация - 1 vCPU, 1 ГБ оперативной памяти, 10 ГБ дискового пространства.

Состав программных средств разработки приведён в таблице 1.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Средство	Назначение
Kotlin	язык программирования клиентской и серверной частей
Jetpack Compose	декларативное построение интерфейса Android-приложения
Room	хранение данных на устройстве поверх SQLite
Hilt	внедрение зависимостей в клиентской части
Ktor Client	обмен с сервером и с реестром маркировки
CameraX и ML Kit	работа с камерой и распознавание кодов DataMatrix и QR
WorkManager	фоновая синхронизация и ежедневный проход по напоминаниям
Material 3	дизайн-система интерфейса
Spring Boot	платформа серверной части
Spring Security	проверка подлинности запросов по ключу устройства
Exposed	составление SQL-запросов к базе данных сервера
PostgreSQL	хранение данных серверной части
Testcontainers	проверка серверной части на настоящей СУБД
Docker и Docker Compose	развёртывание серверной части
Python	язык скраппера справочника препаратов

Таблица 1. Состав программных средств

3.3.2. Обоснование выбора технических и программных средств

Kotlin выбран основным языком обеих частей. Компания Google объявила Android Kotlin-first, поэтому клиентская часть на этом языке получает лучшую поддержку инструментов и библиотек. Тот же язык на сервере позволяет переносить между частями способ описания данных и единообразно рассуждать о границах типов.

Jetpack Compose избавляет от разметки в XML и от ручной синхронизации состояния с элементами интерфейса: экран описывается как функция от состояния.

Room выбран для хранения на устройстве: он проверяет SQL-запросы на этапе компиляции, умеет отдавать результаты потоком и переживает переносы схемы. Программа обязана работать без сети, поэтому местная база - не кэш, а основное хранилище.

ML Kit выбран для распознавания кодов вместо библиотеки ZXing: последняя находится в режиме поддержки и не получает новых возможностей, тогда как ML Kit развивается, лучше распознаёт коды под углом и при плохом освещении и нативно связан с CameraX.

WorkManager выбран для фоновых задач, потому что гарантирует выполнение работы с учётом режимов энергосбережения Android - Doze и App Standby. Напоминание, потерянное из-за режима энергосбережения, обесценивает весь смысл программы.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Spring Boot выбран платформой серверной части. Первоначально рассматривался Ktor как более лёгкий и идиоматичный для Kotlin фреймворк, однако его документация отрывочна, а сообщество существенно меньше, что осложняет и разработку, и изучение. Spring Boot предоставляет зрелую экосистему с единой документацией и большим сообществом - решающий довод при самостоятельной разработке в рамках курсовой работы.

PostgreSQL выбрана СУБД сервера: она надёжна, бесплатна и располагает расширением `pg_trgm`, позволяющим искать препарат по неточному названию через триграммный индекс, без внешнего поискового движка.

3.3.3. Почему состав серверных средств изменился по ходу работы

Состав программных средств серверной части отличается от первоначально заявленного, и это отличие - результат работы, а не смены предпочтений. Изменений два.

Версия платформы. Spring Boot 3 сменился на Spring Boot 4 при обычном обновлении зависимостей. Довод в пользу самой платформы не изменился: она выбиралась за зрелость экосистемы и качество документации.

Отказ от ORM. Первоначально предполагалось использовать Spring Data JPA и Hibernate - стандартную для экосистемы Spring связку объектно-реляционного отображения. По ходу разработки от неё пришлось отказаться в пользу библиотеки Exposed, которая составляет SQL-запросы явно и не пытается скрыть базу данных за объектной моделью. Причин две, и обе выяснились в процессе разработки.

Первая причина - **потеря контроля над формой запроса**. Объектно-реляционное отображение строит запросы само и скрывает как их текст, так и момент обращения к базе. Для обычной прикладной задачи это и есть польза ORM, но в настоящей программе форма запроса - часть требований: в нём выражены доступ вызывающего, предикат версии и режим блокировки, а число запросов ограничено (п. 3.4.8). Требование, выраженное в тексте запроса, невозможно обеспечить средством, которое текст запроса прячет.

Наиболее заметно это проявляется на чтениях. Содержимое аптечки читается вместе с бронями и справочными значениями, и при ленивой загрузке обход полученной коллекции превращается в отдельный запрос на каждую упаковку - задача N+1. Устраняется она подсказками `JOIN FETCH` и `@EntityGraph`, то есть тем же описанием запроса, только косвенным и нечитаемым: разработчик по-прежнему обязан держать форму запроса в голове, но больше не видит её в коде. Проверить испытанием, что число обращений не выросло, в таком виде тоже трудно.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Столь же плохо ORM сочетается с условием доступа. Переход по связи объектов - например, от упаковки к её аптечке и участникам - не несёт предиката вызывающего, поэтому либо доступ проверяется после выборки, что противоречит принятому правилу (п. 3.4.5), либо запрос всё равно пишется руками. Команды добавляют к этому необходимость точно управлять моментом записи и режимом блокировки, тогда как Hibernate решает сам, когда выполнить накопленные изменения.

В итоге поведение JPA и Hibernate приходилось задавать явно почти на каждом шаге, поскольку значения по умолчанию не соответствовали предметной области, и выигрыш от автоматического отображения исчезал: работа по описанию запросов оставалась, а прочитать полученный запрос в коде было нельзя.

Вторая причина - **несовместимость ORM с идиомами языка Kotlin**. Hibernate требует, чтобы поля сущностей были открытыми и изменяемыми, поскольку подменяет их значения во время выполнения через прокси и ленивую загрузку. Это прямо противоречит тому, как на Kotlin описываются данные: неизменяемые `val`, классы данных `data class`, значения-объекты без пустого конструктора. Чтобы удовлетворить требованиям Hibernate, сущности пришлось бы описывать изменяемыми классами с обнуляемыми полями и пустым конструктором, то есть отказаться от тех средств языка, которые делают неверное состояние невыразимым. Кроме того, такое описание вводит в заблуждение: `var` в Kotlin объявляет намерение изменять значение, и у сущности ORM это намерение ложно - значение меняет не программа, а библиотека. От этого страдают и средства статического анализа, и читатель кода.

Exposed снимает обе трудности. Запрос пишется явно и читается как SQL, поэтому число обращений к базе видно прямо в коде сценария, а план запроса можно проверить тестом. Сущности остаются обыкновенными неизменяемыми классами Kotlin, не связанными требованиями библиотеки. Цена решения - отсутствие автоматического отображения объектов на таблицы, то есть необходимость писать преобразование строк в объекты вручную. К моменту перехода эта работа и так выполнялась вручную, поскольку отображение по умолчанию задачам предметной области не отвечало.

3.4. Описание и обоснование архитектуры программы

3.4.1. Общее устройство

Программа разделена на три части, каждая со своей причиной меняться: Android-приложение, серверная часть и скраппер справочника. Клиент обращается к серверу по HTTP, сервер читает

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

справочник из своей базы данных, скраппер наполняет справочник мимо приложения и после этого в работе не участвует.

Часть	За что отвечает	Хранение
Android-приложение	учёт аптек и упаковок, курсы лечения, напоминания, сканирование, очередь операций	Room на устройстве
Серверная часть	общие аптечки, брони, справочник, разрешение конкурентных изменений	PostgreSQL
Скраппер справочника	сбор справочника лекарственных препаратов	файлы загрузки

Таблица 2. Части программы и их ответственность

Обе основные части построены по предметно-ориентированному проектированию (DDD), и в этом проекте оно чаще всего выражается одним правилом: **предметная область не зависит ни от способа хранения, ни от способа доставки**. Правило предметной области - например, «доза берётся из одной упаковки» или «бронь принадлежит человеку, а не серверу» - записано в типе, который не знает ни о таблицах, ни о протоколе передачи. Это позволяет менять хранение и протокол, не затрагивая правил, и проверять правила без базы данных и сети.

3.4.2. Архитектура Android-приложения

Клиентская часть разделена на корни по роду решаемых задач, и направление зависимостей между ними задано жёстко (таблица 3).

Корень	Содержание
domain	предметная область: упаковка, аптечка, курс лечения, приём, величины количества, дозы и срока годности
storage	хранение на устройстве: таблицы Room, запросы, переносы схемы, доменные проекции
network	обмен с сервером и с реестром маркировки: формы запросов и ответов, клиенты
queue	очередь отложенных операций: команда, её состояние, подготовленный запрос, исход
feature	сценарии: целое действие человека, выполняемое одной транзакцией
presentation	подготовка данных к показу, разбор введённого, состояния экранов
ui	экраны и общие составляющие интерфейса
platform	средства операционной системы: уведомления, будильники, разрешения, настройки

Таблица 3. Корни клиентской части

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Предметная область не знает ни об Android, ни о Room, ни о сети; время она получает извне. Очередь операций видит сеть и предметную область, но не видит хранения; сеть о существовании очереди не знает вовсе. Экран и подготовка данных к показу не обращаются ни к базе данных напрямую, ни к сети: сохранённые сведения экран наблюдает потоком из хранилища, а несохранённый ввод остаётся состоянием формы.

Соблюдение этих границ проверяется автоматически: отдельная проверка читает дерево исходных текстов и убеждается, что ни один корень не обращается к тому, чего ему знать не положено. При нарушении сборка завершается с ошибкой, в которой указаны файл и недопустимая зависимость.

Действие человека выполняется **сценарием** - единицей корня feature. Сценарий открывает транзакцию, перечитывает нужные ему объекты, применяет к ним переход предметной области и записывает результат. Сущность действительна только в пределах транзакции, которая её прочитала, поэтому переход применяется к текущему состоянию базы, а не к состоянию, прочитанному экраном ранее. Это снимает целый класс ошибок совместного доступа и одновременно отделяет логику интерфейса от логики работы программы.

Обмен с сервером устроен как **очередь операций**. Действие человека сначала применяется к местной базе, а затем ставится в очередь как команда с подготовленным запросом. Работник очереди отправляет команды по порядку, разбирает исходы и повторяет то, что не доехало. Исход неизвестен - повтор идёт тем же запросом; сервер отверг по версии - очередь перечитывает состояние и готовит запрос заново под тем же номером. Человек при этом не ждёт сети: экран показывает пометку «ещё не доехало» и остаётся работоспособным.

3.4.3. Архитектура серверной части

Серверная часть разделена на слои, обращающиеся строго сверху вниз: контроллеры → прикладные сервисы → оркестраторы сценариев → сервисы агрегатов → хранилища. Как и на клиенте, границы слоёв проверяются программой - по байт-коду и сигнатурам, - и нарушение роняет сборку.

Выделены три ограниченных контекста:

1. **Аптечка** - основной: упаковки, брони, аптечки, участники, пользователи.
2. **Справочник** - данные внешнего скраппера. Только для чтения, наполняется мимо приложения, имеет свою причину меняться.
3. **Журнал приёмов** - отдельный контекст, и причина тому не техническая (п. 3.4.4).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Общими для первых двух контекстов остаются единица измерения и форма выпуска: единица обязана быть одной на систему, иначе «шт» из справочника и «шт» из упаковки оказались бы разными вещами и количества перестали бы складываться.

Корнями агрегатов выбраны упаковка, бронь, аптечка и пользователь. Идентификаторы придумывает **клиент**, а не сервер: это позволяет завести упаковку без сети и отправить её позже под тем же номером, а повторную отправку - распознать как повтор, а не как вторую упаковку.

3.4.4. Приватность как архитектурное ограничение

Требование технического задания - не обрабатывать персональные данные - определило устройство серверной части сильнее любого функционального требования, и его исполнение оказалось существенно сложнее, чем предполагалось при составлении технического задания. Принцип *privacy by design* выражен в следующих решениях:

1. **О человеке хранятся только идентификатор и хеш ключа** - ни имени, ни почты, ни телефона. Учётная запись заводится самим устройством при первом запуске, и связать её с личностью сервер технически не может.

2. **Журнал приёмов не попадает в базу данных сервера.** Сведения о том, кто и когда принял лекарство, слишком персональны, чтобы лежать в таблице: по ним восстанавливается распорядок дня и характер заболевания. Сегодня журнал повторов живёт в памяти процесса, не переживает перезапуск и не разделяется между экземплярами; это осознанное ограничение, названное в документации. Обезличенная квитанция в базе его бы не сняла: таблица всё равно накапливала бы частоту действий, а на домашнем сервере с несколькими участниками такие сведения близки к самому журналу приёмов.

3. **Недоступное неотличимо от несуществующего.** Запрос к чужой аптечке получает тот же ответ, что и запрос к несуществующей: иначе по коду ответа перебором устанавливается, какие аптечки существуют.

4. **Промышленная конфигурация не пишет журналы приложения и обращений.** Диагностическое журналирование остаётся только в местной конфигурации разработчика, поскольку журнал обращений сам по себе накапливает сведения о частоте действий человека.

3.4.5. Доступ: чтение и есть проверка

Решения выше оказались не бесплатны. Эти требование приватности превратило обычную задачу разграничения доступа в самую сложную часть серверной программы.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Обычный способ разграничить доступ - отдельный шаг «проверить право» перед работой с данными. В программе, которая не хранит о человеке ничего, кроме членства в аптечках, такой шаг оказался бы и лишним обращением к базе, и источником расхождений. Поэтому принято правило: **каждое чтение, отдающее данные наружу, принимает вызывающего**, и доступ выражен либо предикатом членства в самом запросе, либо ключом строки.

Агрегат	Чем держится доступ
Бронь	ключом: составной внешний ключ ведёт на членство, поэтому брони без доступа не существует
Упаковка	предикатом: аптечка упаковки входит в аптечки вызывающего
Аптечка	предикатом: соединением с собственным членством вызывающего

Таблица 4. Как выражен доступ к агрегатам

Отдельного шага «проверить доступ» нет: чтение и есть проверка, и без вызывающего запрос не составляется.

Сложность здесь в том, что доступ к общей аптечке отзывается в любой момент: участник выходит из неё или удаляет её, пока другой участник выполняет команду. Прочитанный объект подтверждает только то, что доступ был на момент чтения, и использовать его как разрешение на запись нельзя: между чтением и записью членство может исчезнуть, и тогда запись нарушит внешний ключ вместо того, чтобы получить понятный отказ. Поэтому чтение и команда устроены по-разному: чтение принимает вызывающего и ограничивается запросом, а команда принимает идентификаторы, сама удерживает доступ на время выполнения и уже под ним пересчитывает изменяемое состояние.

3.4.6. Конкурентность и целостность

Общая аптечка означает, что одну и ту же упаковку одновременно меняют несколько человек с разных устройств, часть из которых работает без сети и присылает накопленное позже. Это главный источник сложности серверной части. Четыре вопроса, которые легко свалить в один механизм, разведены намеренно (таблица 5).

Вопрос	Что означает	Чем отвечаем
Видимость	что человеку вообще показывать	предикат членства в запросе
Авторизация команды	вправе ли он менять - и всё ещё ли вправе к концу команды	удержание корневой строки аптечки
Целостность	не разошлись ли связанные строки	внешние ключи как страховка
Версия	не затёр ли он чужое состояние	предикат версии в самом UPDATE

Таблица 5. Четыре механизма и их назначение

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Оптимистическая версия защищает состояние, которое клиент действительно получил, - упаковку или картину броней на ней. Предикат версии стоит в самом запросе изменения: ноль задетых строк означает, что гонку проиграли. Для записи версионизируемого состояния предъявленная версия обязательна.

Удержание корня аптечки отвечает на вопрос об авторизации команды. Используются два режима блокировки:

- **исключительный** - у вступления, выхода, удаления аптечки и переноса упаковки: они меняют состав участников либо место упаковок. Когда затронуты две аптечки, корни берутся в устойчивом порядке идентификаторов - иначе два встречных переноса образуют взаимную блокировку;

- **совместимый** - у всех остальных команд записи: заведения и правки упаковки, приёма, уничтожения, синхронизации, работы с бронью и выдачи приглашения. Совместимые держатели не задерживают друг друга, поэтому обычная работа не выстраивается в очередь, но с жизненным циклом аптечки они упорядочиваются в обе стороны.

Удержание берётся до первой записи и внутри команды не повышается: переход от совместимого режима к исключительному создал бы взаимную блокировку. Под удержанием изменяемое состояние перечитывается, поскольку за время ожидания упаковка могла перейти в другую аптечку или быть удалена.

Отдельной тонкостью оказалась работа механизма согласованного чтения PostgreSQL: блокирующий запрос берёт снимок до ожидания, а параллельный выход участника трогает строку членства, но не саму строку аптечки. Поэтому присоединённая строка членства возвращается из устаревшего снимка, и членство приходится проверять **вторым запросом** - он уже видит зафиксированные изменения, которых ждали. Один запрос вместо двух выглядит экономией, но пропускает именно то состояние гонки, ради которого проверка и выполняется.

Схожая тонкость - выход участника из общей аптечки. Каскадного удаления по внешнему ключу недостаточно: оно убирает строки броней, но не пересчитывает суммы и версии картины броней. Поэтому брони снимает сам сценарий - под исключительным удержанием корня, в том же порядке, в каком их удерживают изменение и отмена брони, - а каскад остаётся страховкой целостности, а не механизмом. Триггеров в базе данных нет: правила остаются в коде, а ключи страхуют.

Коды ответов различают исходы так, чтобы клиент мог действовать без догадок: версия не прислана - 428, нечитаема - 400, предъявлена и не совпала - 412, недоступно или не существует - 404,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

а 409 остаётся конфликтам, предусловием запроса не бывшим: повторное вступление, дубль брони и занятый клиентский идентификатор с другим телом.

3.4.7. Синхронизация клиента и сервера

Согласование клиента и сервера потребовало наибольшей переработки модели. Причина в том, что требование приватности запрещает передавать серверу журнал приёмов, а без него сервер не знает ни состава, ни порядка операций, выполненных на устройстве. Согласование строится на трёх положениях.

1. Клиент копит события без сети и присылает суммы. Съеденное и новая бронь - два разных состояния одной упаковки, и порядок между двумя отдельными запросами не гарантирован: сначала приходит списание, срабатывают предупреждения «лекарства мало», и лишь затем уменьшение брони, которое эти предупреждения снимает.

2. Поэтому синхронизация выполняется **единым запросом, атомарным на упаковку**. Съеденное передаётся приращением - оно коммутативно и не требует предусловия, - а бронь абсолютным значением, поскольку она и есть решение владельца. Только в этом запросе версии переезжают из параметра в тело: он меняет два состояния сразу.

3. Журнал повторов у приёма и синхронизации общий и пишется **после фиксации транзакции**: запись по ходу команды пережила бы откат, и повтор получил бы подтверждение того, чего в базе нет. Спрашивается журнал раньше предиката версии - поэтому повтор потерянного ответа не получает отказ по версии, даже если она с тех пор ушла вперёд.

3.4.8. Описание работы с базами данных

В программе две базы данных: местная на устройстве и серверная.

Местная база хранит всё, чем человек пользуется без сети: аптечки, упаковки, вечные записи об упаковках, курсы лечения и их источники, приёмы, обязательства по напоминаниям, очередь операций и снимок справочника. Ограничения вынесены в саму схему, а не оставлены коду: активное назначение упаковки под курс уникально, количества неотрицательны, а связанные строки удаляются согласованно. Схема переносится версионированными миграциями; каждая миграция проверяется отдельно - как на сохранность данных, так и на совпадение полученной схемы с описанной.

Серверная база хранит пользователей, аптечки, членство, упаковки общих аптек, брони и справочник препаратов. Таблицы `form_types` и `quantity_units` общие для контекста аптечки и контекста справочника.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

К запросам серверной части предъявляются четыре требования, и все они относятся не к результату запроса, а к его **форме**.

1. **Доступ выражен внутри запроса.** Чтение отдаёт только то, что видно вызывающему, поэтому предикат членства входит в сам запрос, а не применяется к полученным строкам (п. 3.4.5). Фильтрация после выборки означала бы, что чужие строки уже прочитаны, а условие доступа существует отдельно от запроса и может быть пропущено.

2. **Версия проверяется тем же запросом, который изменяет строку.** Предикат версии стоит в UPDATE, и признаком проигранной гонки служит ноль изменённых строк (п. 3.4.6). Проверка версии отдельным чтением оставляла бы промежуток между проверкой и записью.

3. **Блокировка берётся точно названной строкой.** Корень аптечки удерживается запросом FOR UPDATE OF med_kits или FOR KEY SHARE OF med_kits, причём удержание не должно распространяться на присоединённую строку членства, иначе исключительный режим конфликтует с внешними ключами чужих броней (п. 3.4.6).

4. **Число обращений к базе не растёт с объёмом данных.** Содержимое аптечки, снимок, картина броней и команды выполняются постоянным числом запросов независимо от того, сколько в аптечке упаковок и сколько у них броней. Это свойство проверяется испытаниями, а планы выполнения проверяются на выборке в сорок тысяч строк.

Первые три требования означают, что в коде должен быть виден текст запроса, а четвёртое - что должно быть видно и их количество. Обращение к базе идёт явными запросами через Exposed именно поэтому: в коде сценария видно и то, и другое.

3.5. Описание и обоснование выбора метода организации входных и выходных данных

Источники входных данных перечислены в техническом задании: ручной ввод, сканирование кодов камерой, данные синхронизации с сервером и данные справочника препаратов.

Принят следующий метод организации ввода: **разбор введённого человеком не принадлежит предметной области**. Пока человек печатает, в поле лежит «12,», «0.0000001» или пусто - ни одно из этих состояний величиной не является. Приведением введённого к величине занимается слой представления: он возвращает либо готовую величину, либо названную причину отказа, но никогда не бросает исключение. Предметная область принимает только готовые величины и проверяет их смысл: допустимость количества, согласованность единиц, правила валюты, смысл срока годности.

Такое разделение обосновано тем, что способ ввода меняется чаще правил. Появление сканера, подсказок справочника или иного формата ввода не должно затрагивать правила предметной области

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- и не затрагивает: признаком неверно проведённой границы служит необходимость править предметную область при изменении формы ввода.

Заведомо «нереалистичные» данные не запрещаются. Например, программа принимает истёкший срок годности, как того требует техническое задание, и обрабатывает его - помечает упаковку просроченной, предупреждает при попытке принять из неё дозу и показывает её выше остальных.

3.6. Особенности функционала

Ниже описаны особенности реализации функций, перечисленных в п. 4.1.1 технического задания. Интерфейс построен по дизайн-системе Material 3; цвет нигде не является единственным носителем сообщения - рядом с ним всегда стоят текст и значок.

3.6.1. Первый запуск и учётная запись устройства

При первом запуске приложение заводит учётную запись устройства само: придумывает идентификатор и ключ, сохраняет ключ в защищённом хранилище устройства и получает у сервера доступ. Человек не вводит ни логина, ни пароля - их не существует. Экран первичной настройки показывает, на каком шаге находится подготовка, и предлагает повторить при отсутствии связи (рис. 1).

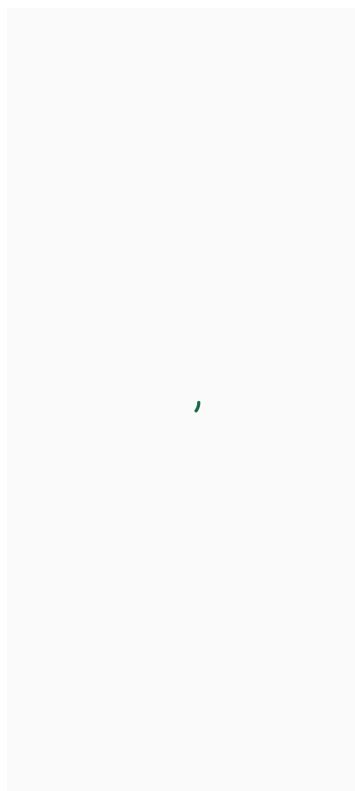


Рис. 1. Первичная настройка

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.6.2. Аптечки и их содержимое

Список аптечек - первое, что человек видит. Каждая строка показывает число упаковок, число просроченных и признак общей аптечки с количеством участников (рис. 2). Содержимое аптечки показывает упаковки с остатком, сроком годности и пометками состояния; просроченные упаковки выделены и стоят выше остальных, как того требует п. 4.2 технического задания (рис. 3).

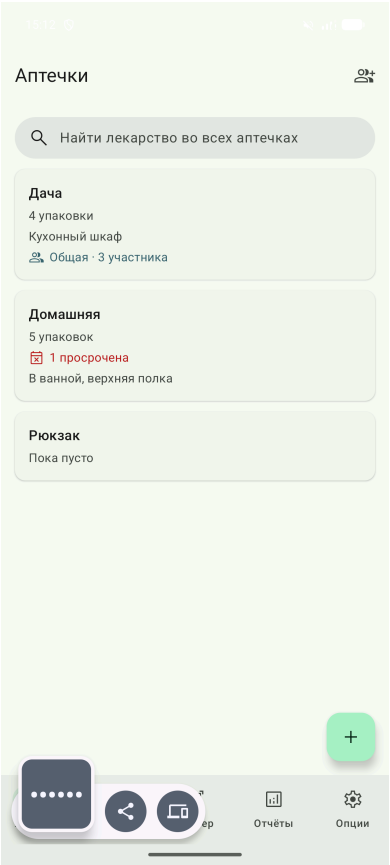


Рис. 2. Список аптечек

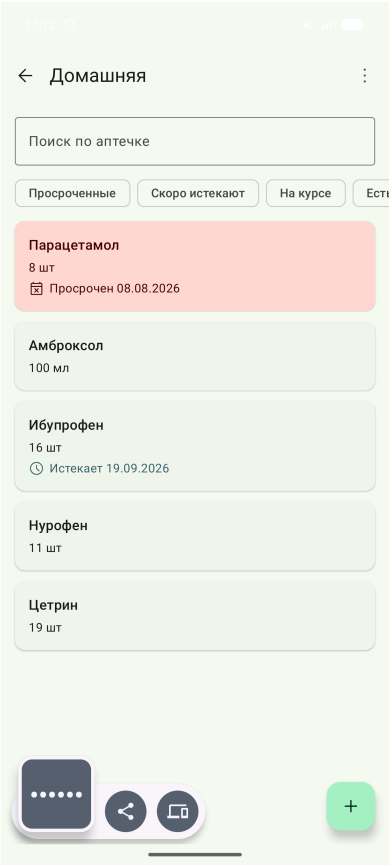


Рис. 3. Содержимое аптечки

Поиск, фильтрация по одному параметру и сортировка применяются в любом порядке и совместно; результат не зависит от последовательности действий (рис. 4). Поиск ведётся как в выбранной аптечке, так и по всем доступным сразу.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

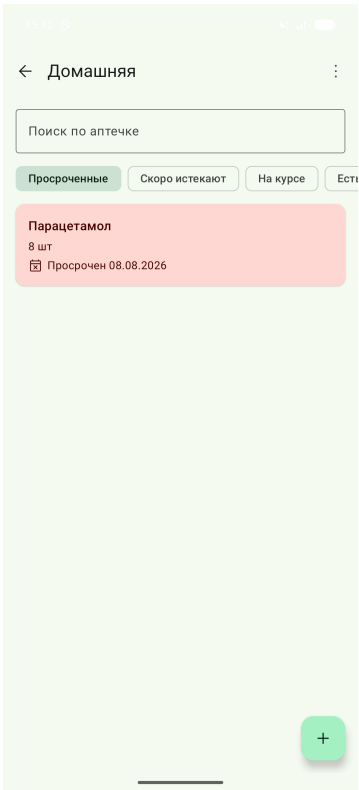


Рис. 4. Сужение списка поиском и фильтром

3.6.3. Карточка упаковки

Карточка упаковки построена от срочного к справочному: сколько осталось, до каких пор годно, где лежит - и лишь затем сведения из справочника. Здесь же показано, занята ли упаковка лечением, есть ли чужие брони и не разошлось ли состояние с сервером. Просроченная упаковка выделена красным, ожидающее ответа сервера решение - янтарным, а выброшенная упаковка показывается «призраком» и не нажимается (рис. 5, 6).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

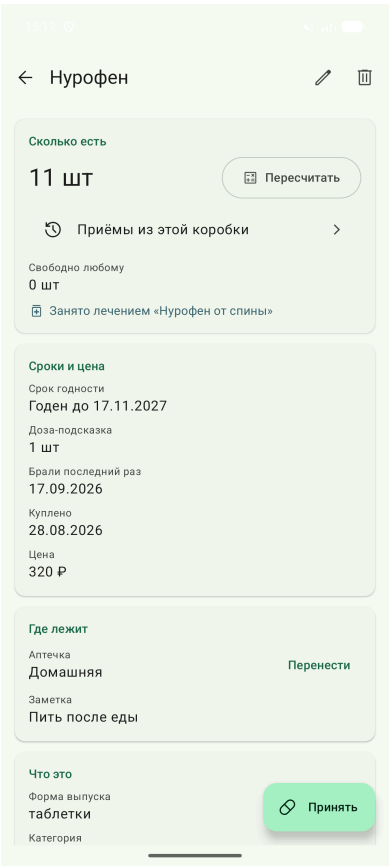


Рис. 5. Карточка упаковки, занятой лечением

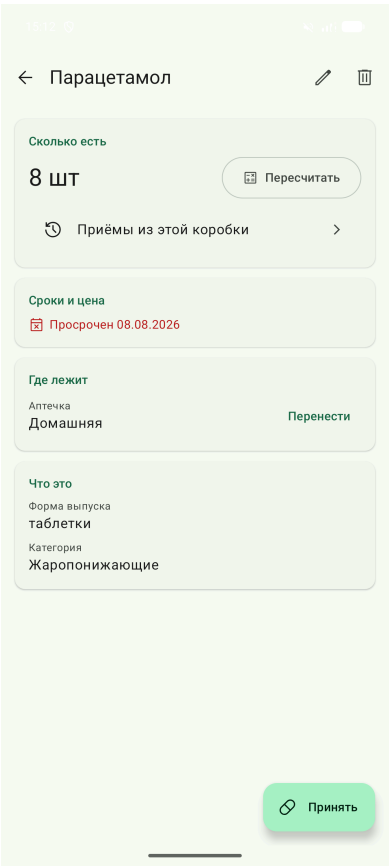


Рис. 6. Просроченная упаковка

3.6.4. Заведение упаковки и сканирование кода

Упаковку заводят вручную либо сканированием кода маркировки. При ручном вводе название подсказывает справочник: поиск ведётся по неточному совпадению, и выбор подсказки заполняет известные поля. Обязательными остаются название, форма, количество и срок годности (рис. 7).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

← Новая упаковка

Аптечка

Домашняя

Название

обязательно

Форма выпуска

Количество

Единица

обязательно

обязательно

Срок годности

Как на упаковке: 03.2027 или 31.03.2027

Категория

Производитель

Страна

Сохранить

Отмена

Рис. 7. Заведение упаковки

Сканер открывается сразу видеискателем - за этим на него и приходят (рис. 8). Формат кода определяет распознаватель: код маркировки ведёт в заполненную форму упаковки, код приглашения - на экран вступления в общую аптечку, прочее - сообщение «этот код не поддерживается».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

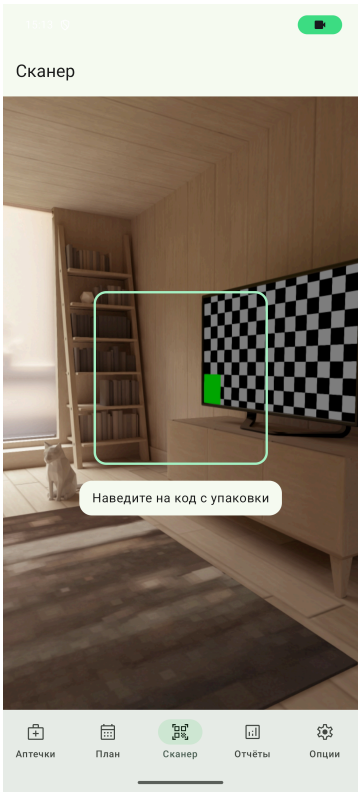


Рис. 8. Сканер кодов

3.6.5. Курсы лечения и их обеспечение

Курс лечения описывается дозой, формой, расписанием по дням недели и временам приёма и числом доз. Список курсов показывает идущие, черновики и завершённые лечения, отмечая нехватку обеспечения (рис. 9). Карточка курса начинается с обеспечения - за ним сюда и приходят, - а далее показывает назначение, источники и пункты плана (рис. 10).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

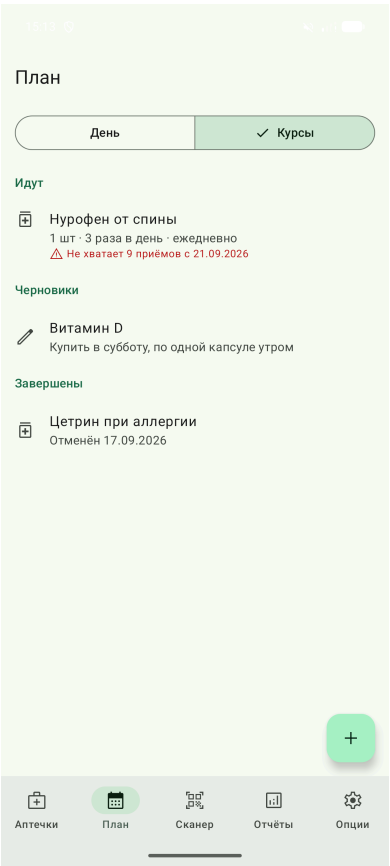


Рис. 9. Список курсов лечения

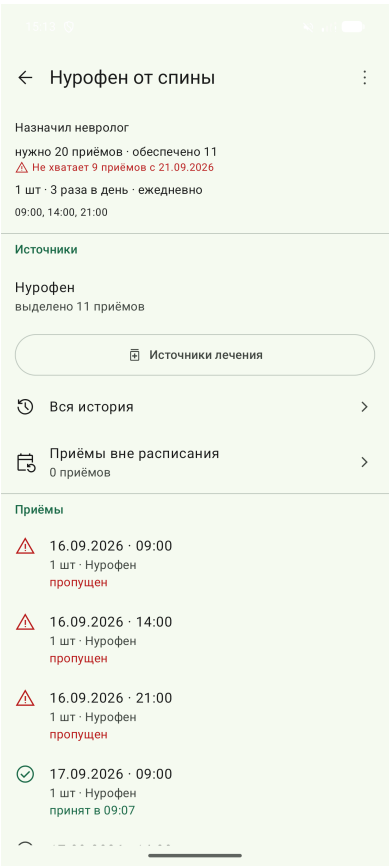


Рис. 10. Карточка курса лечения

Источники курса - упаковки, из которых берутся дозы, в порядке расходования (рис. 11). Здесь видно, сколько доз выделено из каждой упаковки и что мешает подключить ту или иную упаковку: чужая единица измерения, занятость другим лечением или нехватка остатка. Нехватка не сокращает расписание молча: программа сообщает о ней и предлагает способы исправления - подключить ещё упаковку, перераспределить дозы или завести новую покупку.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

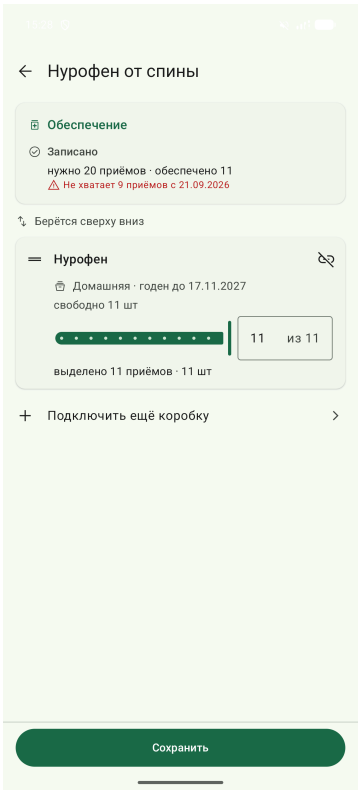


Рис. 11. Источники курса лечения

3.6.6. План дня, приёмы и напоминания

Страница дня показывает назначенные приёмы, их состояние и позволяет ответить одним нажатием - «принял» или «пропустил» (рис. 12). Страница держит смещение от сегодняшнего дня, а не дату: в полночь она сама начинает показывать новый день, и набранное человеком не теряется.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

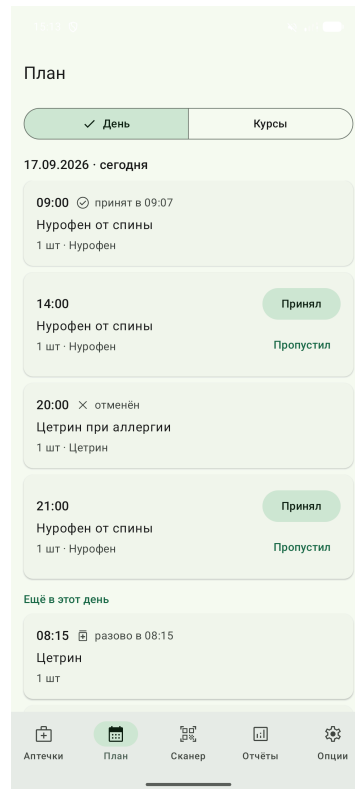


Рис. 12. План дня

Напоминание приходит уведомлением в назначенное время (рис. 13). Ответить можно прямо из шторки, не открывая приложение: у уведомления есть действия «Принял», «Пропустил» и «Отложить», и ответ записывается в фоне. Если сказать не удалось - уведомления запрещены или заглушены, - программа не молчит: неотвеченные приёмы прошлых дней встречаются человека при входе всплывающим окном (рис. 14), а страница дня прямо говорит, почему напоминания не приходят, и ведёт в системные настройки.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

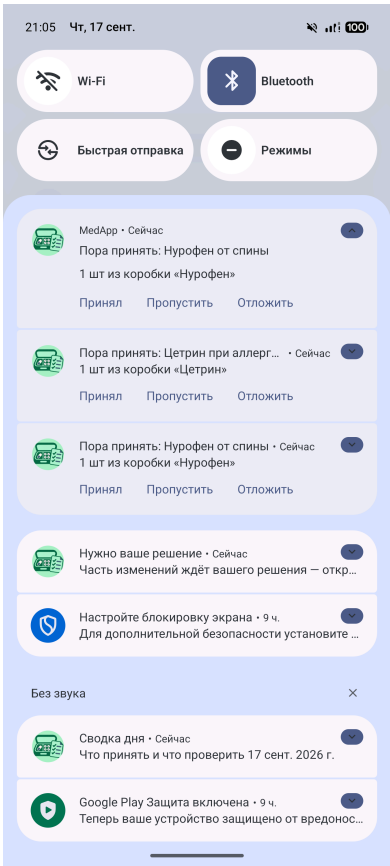


Рис. 13. Напоминание о приёме в шторке уведомлений

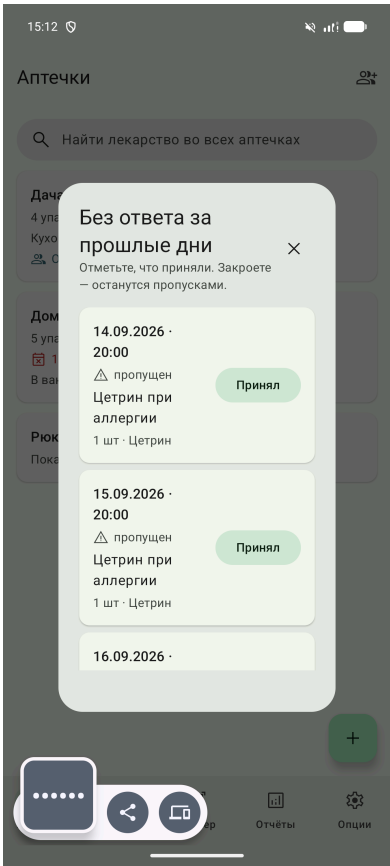


Рис. 14. Приёмы без ответа за прошлые дни

3.6.7. Общая аптечка

Прежде чем выдать приглашение, программа объясняет последствия: сведения об аптечке попадут на сервер, а переданный доступ отозвать нельзя. Приглашение выдаётся кодом и QR-кодом (рис. 15); присоединяются вводом кода или его сканированием (рис. 16).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

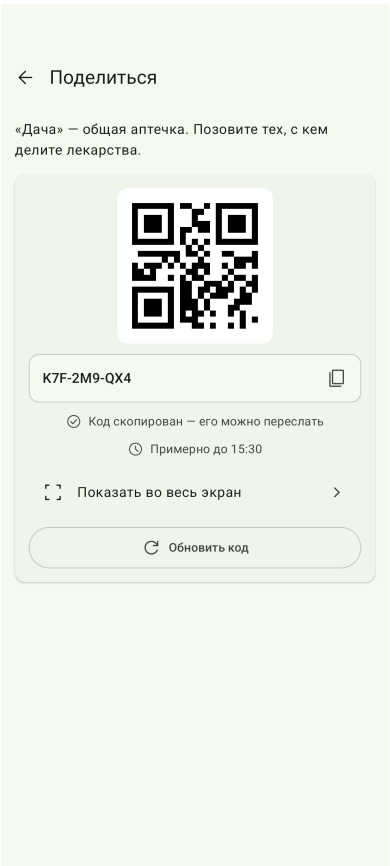


Рис. 15. Общая аптечка и приглашение

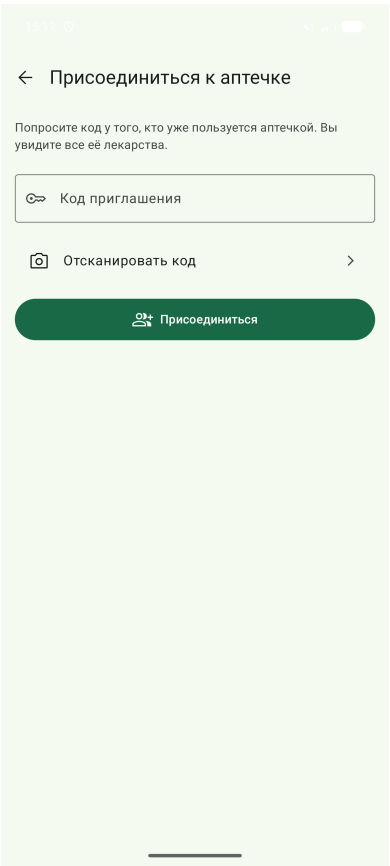


Рис. 16. Присоединение к общей аптечке

Состояние обмена с сервером человеку видно: экран синхронизации показывает, что ещё не доехало, что отвергнуто и по какой причине, и что делать дальше (рис. 17). Расхождение по количеству лечится пересчётом - тем же экраном, которым пересчитывают упаковку в обычной жизни.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

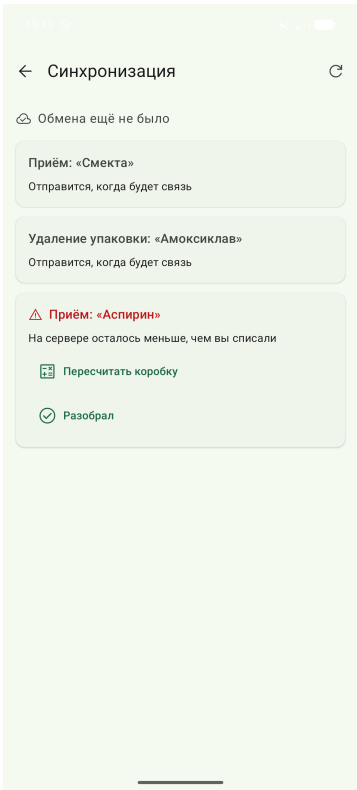


Рис. 17. Состояние синхронизации

3.6.8. Отчёты

Раздел отчётов отвечает на три вопроса п. 4.1.1.10 технического задания, и вопросы эти разной природы, поэтому разложены переключателем, а не свалены в один список.

Сводка отвечает, что у человека есть сейчас: число упаковок, их распределение по категориям и формам и общая цена по валютам. Упаковки, у которых цена не названа, считаются отдельным числом, а не бесплатными: «неизвестно» и «ноль» - разные состояния (рис. 18).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

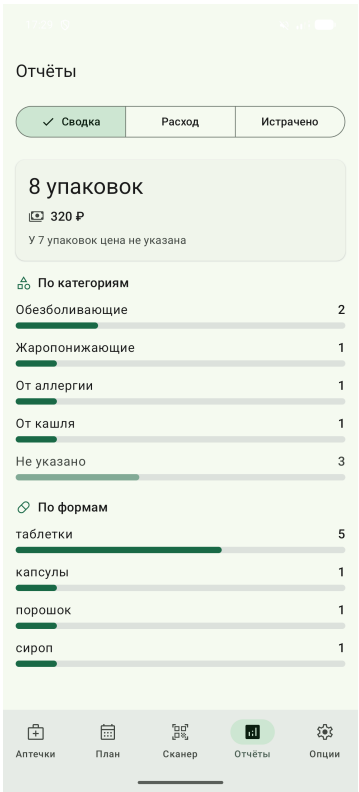


Рис. 18. Сводная статистика

Расход отвечает, сколько уйдёт до выбранной даты, и считается по идущим курсам лечения в предположении, что все приёмы состоятся, - как того требует п. 4.1.1.10.1. Дальше трёх месяцев спросить нельзя: такие дни в календаре не нажимаются (рис. 19).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

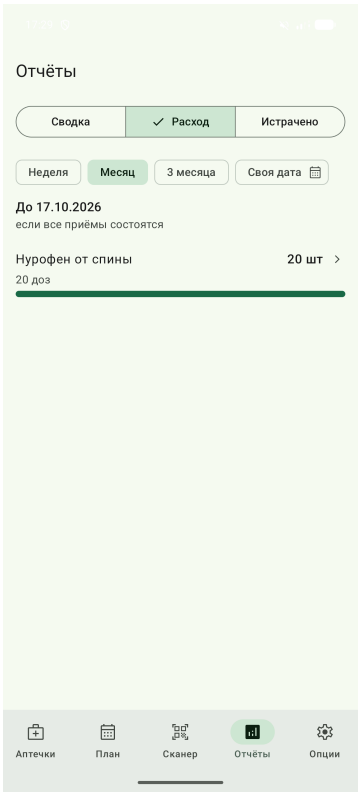


Рис. 19. Расход до даты

Истрачено отвечает, сколько человек принял за период не длиннее года. Здесь уместно пояснить формулу п. 4.1.1.10.2 «все добавленные минус все, что остались»: она описывает механику словами заказчика, а не способ расчёта. Истраченное человеком и есть то, что он принял, поэтому отчёт считается по состоявшимся приёмам, включая приёмы мимо курса. Такой расчёт не путает расход с переносом упаковки между аптечками, с пересчётом остатка и с расходом другого участника общей аптечки - а разностная формула сочла бы расходом всё перечисленное. Завершённое лечение из отчёта не исчезает: запись о нём вечна, и выпитые по нему дозы остаются выпитыми (рис. 20).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

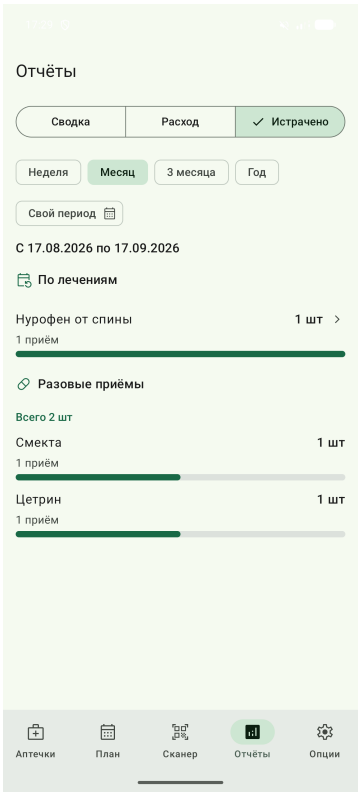


Рис. 20. Истрачено за период

Сводный план лечения на дату (п. 4.1.1.10.4) показывает страница дня, описанная выше: заводить для одного и того же второй экран незачем.

3.6.9. Настройки и разрешения

Настройки собраны в отдельном месте: пороги и время напоминаний, состояние системных разрешений с путём к их исправлению и выбор языка приложения (рис. 21). Программа объявляет русский и английский языки и до выбора человека говорит на языке устройства.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

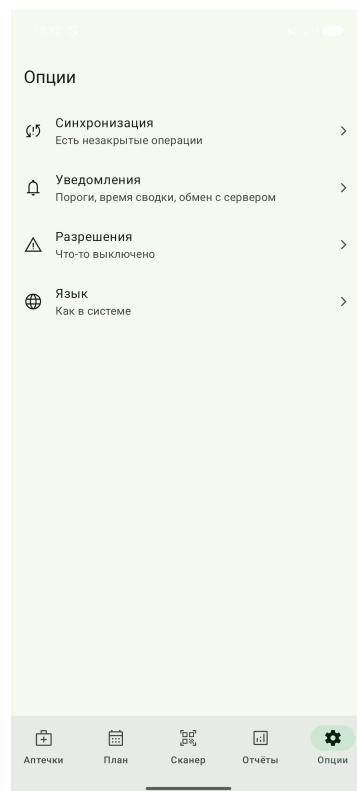


Рис. 21. Настройки приложения

3.7. Организация разработки

Разработка велась одним человеком, поэтому порядок работы задавался не распределением обязанностей, а письменной фиксацией решений: всё, что не записано, при возвращении к задаче восстанавливается заново и, как правило, иначе.

3.7.1. Два уровня планирования

Планирование разделено на два уровня, и у каждого своя задача.

Глобальный план описывает систему целиком: понятия предметной области и их отношения, устройство частей, принятые продуктовые решения с их обоснованием и то, чего в программе сознательно нет. Он отвечает на вопросы, которые не принадлежат ни одной отдельной задаче: что такое упаковка и чем она отличается от лекарства, кому принадлежит бронь, какие сведения не должны покидать устройство. Глобальные планы велись отдельно для серверной части и для клиента и правились при переходе к следующему этапу, а не дополнялись задним числом.

Локальный план составлялся на каждое изменение и отвечал на вопросы одной задачи: что именно меняется, какие проверки это подтверждают, какие решения принимаются по ходу и что остаётся за границами изменения. Локальный план не повторяет глобальный, а ссылается на него.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Такое разделение решает две разные трудности. Без глобального плана каждое изменение начинает переопределять понятия под себя, и предметная область расплзается. Без локального плана изменение разрастается: в него попадает всё, что «заодно заметили», и проверить его целиком становится невозможно.

3.7.2. Прослеживаемость изменений

Каждое изменение вносилось отдельным запросом на слияние (pull request), а не прямой записью в основную ветку. Запрос на слияние содержит описание задачи, состав изменений, перечень выполненных проверок и результаты рецензирования, поэтому история работы над программой восстанавливается по репозиторию: видно, какое решение принято, какими проверками оно подтверждено и какие замечания были сняты до включения изменения.

Замеченное, но выходящее за границы текущего изменения, оформлялось отдельной задачей (issue) и разбиралось позже. Это удерживает изменение в заявленных границах, не теряя находок.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Предполагаемая потребность

Лекарства в домохозяйстве хранятся одновременно в нескольких местах - дома, на работе, на даче, - а часть препаратов принимается курсами и требует регулярности. В таких условиях трудно оперативно установить текущие запасы, понять, что скоро закончится и что уже просрочено, и учесть, сколько препарата зарезервировано под запланированные курсы. Положение усложняет явление «общей аптечки», когда одни и те же лекарства расходуются разными людьми и остаток меняется непредсказуемо.

Целевая аудитория программы:

- люди, проходящие курс лечения или принимающие препараты регулярно;
- семьи и домохозяйства с общей аптечкой;
- люди, у которых несколько мест хранения лекарств;
- люди, желающие снизить потери от просроченных лекарств и повторных покупок за счёт прозрачного учёта и своевременных напоминаний.

4.2. Экономические преимущества разработки по сравнению с аналогами

Экономическая польза программы для человека предполагается в следующем:

1. Уменьшение потерь из-за просрочки: программа предупреждает заранее и показывает просроченные позиции первыми.
2. Снижение повторных покупок: текущие запасы и места хранения видны сразу, и покупка «на всякий случай» перестаёт быть необходимой.
3. Снижение срывов курса из-за нехватки лекарства: расход по идущим курсам считается заранее, а нехватка обеспечения показывается до того, как лекарство кончится.
4. Уменьшение затрат времени на поиск и сверку: единый список и поиск по всем аптечкам заменяют ручной осмотр коробок.

Для сравнения отобраны пять приложений: два зарубежных, наиболее известных в разряде напоминаний о приёме, два российских, ведущих учёт домашней аптечки, и официальное приложение системы маркировки, поскольку оно решает ту же задачу распознавания упаковки:

- **Medisafe** [24] - зарубежное приложение напоминаний о приёме с семейными профилями и напоминанием о пополнении запаса; расширенные возможности платные;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- **MyTherapy** [25] - зарубежное бесплатное приложение напоминаний с ведением нескольких профилей и дневником состояния;

- «**Домашняя аптечка**» [26] - российское приложение с открытым исходным текстом: добавление лекарств сканированием кода, сортировка по срокам годности, напоминания и облачная синхронизация списка;

- «**Аптечка. Учёт медикаментов**» [27] - российское приложение, ведущее несколько аптек (домашнюю, автомобильную, детскую) с контролем сроков и проверкой упаковки на признаки подделки;

- «**Честный ЗНАК**» [28] - официальное приложение системы маркировки: распознаёт код DataMatrix, показывает сведения об упаковке и её срок годности, содержит будильник приёма, но учёта домашних запасов не ведёт.

Сравнение приведено в таблице 6. Оно выполнено по открытым описаниям приложений в магазинах и на сайтах разработчиков по состоянию на сентябрь 2026 года; запись «не заявлено» означает, что возможность в описании не указана.

Возможность	Medisafe	My-Therapy	Домашняя аптечка	Аптечка. Учёт	Честный ЗНАК	MedApp
Напоминания о приёме	+	+	+	+	+	+
Учёт остатка упаковки	+	+	+	+	-	+
Несколько мест хранения	-	-	-	+	-	+
Контроль сроков годности	-	-	+	+	+	+
Сканирование кода маркировки	-	-	+	+	+	+
Справочник препаратов РФ	-	-	+	-	+	+
Общая аптечка с общим остатком	-	-	-	-	-	+
Бронирование под курс лечения	-	-	-	-	-	+
Полная работа без сети	-	+	+	+	-	+
Без учётной записи и персональных данных	-	-	-	-	-	+
Стоимость	подписка	бесплатно	бесплатно	бесплатно и PRO	бесплатно	бесплатно

Таблица 6. Сравнение с отечественными и зарубежными аналогами

Из таблицы видно, что возможности разбиваются на две группы. Напоминания, учёт остатка и контроль сроков годности в том или ином виде есть у всех приложений учёта, и по ним программа не предлагает ничего принципиально нового.

Отличия сосредоточены в трёх местах.

1. **Совместное использование одной аптечки.** Ни одно из рассмотренных приложений не позволяет нескольким людям вести общий запас: семейные возможности Medisafe и MyTherapy - это

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

несколько профилей приёма в одной учётной записи, где у каждого человека свой список лекарств. Общего остатка, который расходуют разные люди с разных устройств, в них нет, а именно он и описывает явление домашней аптечки.

2. Бронирование остатка под курс лечения. Следствие первого: если запас общий, то назначенное лечение должно закреплять за собой часть остатка, иначе другой участник израсходует препарат, и лечение прервётся. Такой возможности нет ни у одного из аналогов.

3. Отсутствие учётной записи и персональных данных. Зарубежные приложения требуют регистрации и хранят сведения о приёмах на своей стороне. В настоящей программе человек не сообщает о себе ничего: учётная запись заводится устройством, а сервер хранит только идентификатор и хеш ключа (п. 3.4.4).

Экономический смысл этих отличий тот же, что и у перечисленных выше выгод: общий учёт избавляет домохозяйство от повторных покупок «на всякий случай» и от срыва лечения из-за того, что препарат, на который человек рассчитывал, израсходован кем-то другим.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ

При разработке программы и подготовке программной документации применялись генеративные модели. Настоящий раздел описывает цели их применения, наименования моделей, адреса их размещения в сети «Интернет» и способы применения.

5.1. Цели применения

1. Рецензирование исходного текста программы: поиск ошибок, несогласованности и нарушений принятых в проекте правил перед включением изменений в основную ветку.
2. Генерация исходного текста программы по сформулированным автором требованиям с последующей проверкой и правкой.
3. Подготовка черновиков программных документов с последующей проверкой, правкой и дополнением автором.

5.2. Применявшиеся модели

Claude (Anthropic), версии Opus 5, Opus 4.6 и, в незначительном объеме, Fable [20]. Применялась через интерфейс командной строки Claude Code [21] в репозитории проекта. Способ применения: генерация исходного текста программы по сформулированным автором требованиям, рецензирование подготовленных изменений и подготовка черновиков программных документов, включая настоящую пояснительную записку.

GPT (OpenAI), версии Astra, 5.5 и Sol [22]. Способ применения: рецензирование подготовленных изменений и подготовка отдельных небольших фрагментов исходного текста программы.

CodeRabbit [23] - средство автоматического рецензирования, встроенное в процесс работы с запросами на слияние. Способ применения: разбор каждого запроса на слияние перед включением изменений в основную ветку. Замечания разбирались автором: обоснованные устранялись, ошибочные отклонялись с указанием причины в самом обсуждении.

5.3. Способ применения и разграничение ответственности

Модели применялись как вспомогательное средство при сохранении авторского контроля над результатом. Постановка задачи, архитектурные решения, выбор технических и программных средств, правила предметной области и окончательная редакция всех материалов принадлежат автору работы.

Порядок применения был следующим. Автор формулировал требование к изменению и ограничения, которым оно обязано удовлетворять; модель предлагала реализацию; автор проверял её на соответствие требованиям, правилам проекта и результатам испытаний, после чего принимал,

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

правил или отвергал предложенное. Каждое изменение проходило автоматические проверки границ и поведения, описанные в пп. 3.4.2 и 3.4.3, и рецензирование; изменения, не прошедшие проверки, в основную ветку не включались.

Черновики программных документов, подготовленные с применением моделей, проверялись автором на соответствие действительному поведению программы, требованиям технического задания и требованиям ГОСТ 19, после чего правились и дополнялись.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. - М.: ИПК Издательство стандартов, 2001.
10. Приложение для Android «Организатор лекарств». Техническое задание. RU.17701729.06.05-01 ТЗ 01-1.
11. Kotlin Documentation. // JetBrains. URL: <https://kotlinlang.org/docs/home.html>
12. Jetpack Compose. // Android Developers. URL: <https://developer.android.com/compose>
13. Room Persistence Library. // Android Developers. URL: <https://developer.android.com/training/data-storage/room>
14. WorkManager. // Android Developers. URL: <https://developer.android.com/topic/libraries/architecture/workmanager>
15. Material Design 3. URL: <https://m3.material.io>
16. Spring Boot Reference Documentation. // VMware. URL: <https://docs.spring.io/spring-boot/index.html>
17. Exposed: Kotlin SQL Framework. // JetBrains. URL: <https://www.jetbrains.com/help/exposed/home.html>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

18. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/>
19. Adherence to long-term therapies: evidence for action. // World Health Organization, 2003.
20. Claude. // Anthropic. URL: <https://claude.ai>
21. Claude Code. // Anthropic. URL: <https://claude.com/claude-code>
22. ChatGPT. // OpenAI. URL: <https://chatgpt.com>
23. CodeRabbit. URL: <https://www.coderabbit.ai>
24. Medisafe Pill & Med Reminder. // Google Play. URL: <https://play.google.com/store/apps/details?id=com.medisafe.android.client>
25. Meds & Pill Reminder MyTherapy. // Google Play. URL: <https://play.google.com/store/apps/details?id=eu.smartpatient.mytherapy>
26. Домашняя аптечка. // RuStore. URL: <https://www.rustore.ru/catalog/app/ru.application.homemedkit>
27. Аптечка. Учёт медикаментов. // RuStore. URL: <https://www.rustore.ru/catalog/app/ru.decsoft.myaidkit>
28. Честный ЗНАК: проверка товаров. // Официальный сайт системы маркировки. URL: <https://markirovka.ru>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ТЕРМИНОЛОГИЯ

Термин	Значение
Комплаентность	степень соответствия между поведением пациента и рекомендациями врача
Аптечка	набор лекарств, сосредоточенный в одном месте
Общая аптечка	аптечка, доступная нескольким людям
Упаковка	физическая пачка лекарственного средства со своим сроком годности и остатком
Форма выпуска	вид, в котором распространяется лекарство: таблетки, порошок, жидкость и т. д.
Единица учёта	единица, в которой считается количество: штуки, миллилитры, граммы
Приём	употребление определённого количества препарата
Разовый приём	приём, не входящий в курс лечения
Курс лечения	назначенная последовательность приёмов с дозой, расписанием и числом доз
Источник курса	упаковка, из которой курс берёт дозы
Обеспечение	число доз курса, покрытое его источниками
Бронь	часть остатка общей упаковки, закреплённая за курсом одного из участников
Очередь операций	местный список действий, ожидающих отправки на сервер
Справочник	база сведений о лекарственных препаратах, наполняемая скраппером
Код маркировки	код DataMatrix на упаковке, зарегистрированный в системе «Честный знак»

Таблица 7. Терминология

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.05-01 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]